Repo clone na kore jodi zip download kora hoy tahole alada vabe init kore nite hobe.

git init
git add .
git commit -m "task-1"

Compilation:
javac EmployeeManager.java

Branch:
- First branch (Master korar jonno): git branch -m master
- Branch creation: git checkout -b task-1
- Total branch checking: git branch
- Ek branch theke onno branch: git checkout [branch name]

`git add` আপনার কোডের পরিবর্তনগুলোকে সেভ করার জন্য রেডি (Stage) করে, আর **`git commit`** সেগুলোকে একটি মেসেজসহ গিট-এর হিস্ট্রিতে স্থায়ীভাবে সেভ (Save) করে রাখে।

Merging abort korte hole : git merge --abort
Conflict ashle:
git add EmployeeManager.java
git commit -m "Resolved merge conflict"

Task 1: Update Code Style

Style update: Shift + Alt + F

```
git checkout -b task-1
git add .
git commit -m "Task #1. Update Code Style for Better Consistency:"
git checkout master
git merge task-1
```

Task 2: Terminate Early on Wrong Args

বর্তমানে যদি কেউ টার্মিনালে `java EmployeeManager` লিখে এন্টার দেয়,
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException
এই এরর আসবে, কারণ কোনো আর্গুমেন্ট (ইনপুট) পাস করেননি। ফলে `args` অ্যারে-টির সাইজ হয়েছে `0`।

solution :
Code:

```
git checkout -b task-2
```

`main` মেথডের একদম শুরুতে এই কোডটি বসান:

```
if (args.length != 1) {
    System.out.println("Error: Wrong number of arguments!");
    return;
```

```
}
```

```
git add .
git commit -m "Task #2. Application Now Terminates Early if the Number of Arguments Passed into It Is
Wrong, Fix It:"
git checkout master
git merge task-2
```

Task 3: Meaningful Variable Names

Code: `StudentList.java` ফাইলে থাকা ছোট ভেরিয়েবল (যেমন: `s`, `i`) গুলোকে রিনেম (F2) করে `line`, `index`, `fileReader`, `employeeName` এর মতো নাম দিন।

`r` → `reader` (ফাইল পড়ার জন্য BufferedReader)

`l` → `line` (ফাইল থেকে পড়া স্ট্রিং লাইন)

`e` → `employees` (কমা দিয়ে ভাগ করা এমপ্লয়িদের স্ট্রিং অ্যারে)

`w` → `writer` (ফাইল লেখার জন্য BufferedWriter)

`n` → ব্লকের কাজের ওপর ভিত্তি করে ৩টি অর্থপূর্ণ নাম দেওয়া হয়েছে:

- **`newEmployee`** (যোগ করার ব্লকে)
- **`targetToUpdate`** (আপডেট করার ব্লকে)
- **`targetToDelete`** (ডিলিট করার ব্লকে)

`s` → `targetName` (সার্চ করার ব্লকে যে নামটি খোঁজা হচ্ছে)

`c` → `character` (লুপের ভেতরের প্রতিটি ক্যারেক্টার)

```
git checkout -b task-3
git add .
git commit -m "Task #3. Makes Improvements to Variable Names:"
git checkout master
git merge task-3
```

Task 4: Refactor File Read/Write

BufferedReader , BufferedWriter বার বার ইউজ হইছে,

Code: ক্লাসের ভেতরে নিচের দুটি মেথড তৈরি করুন এবং মেইন মেথড থেকে এগুলো কল করুন:

```
public static String readFromFile(String fileName) {
    try {
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(new FileInputStream(fileName)));
        return reader.readLine();
    } catch (Exception e) { return ""; }
}

public static void writeToFile(String fileName, String text) {
    try {
        BufferedWriter writer = new BufferedWriter(
            new FileWriter(fileName, true)); // true for append
        writer.write(", " + text);
        writer.close();
    } catch (Exception e) {}
```

```
}
```

Task 5: Constants Class

Code: একই ফোল্ডারে `Constants.java` নামে নতুন ফাইল খুলুন:

```
public class Constants {

    // Constant for the employee file path

    public static final String EMPLOYEE_FILE_PATH = "employees.txt";


    // You can add date format constants here in the future, for example:

    // public static final String DATE_FORMAT = "yyyy-MM-dd";

}
```

এরপর `StudentList.java` ফাইলে ফাইলের নামের জায়গায় `Constants.FILE_NAME` বসান।

```
git checkout -b task-5
git add .
git commit -m "Task #5. Replaces String Literals with Constants, Storing Those Constants in a New Class
Called Constants.java:"
git checkout master
git merge task-5
```

Task 6: Remove Temporary Variables

Code: কোডে যদি এমন কোনো `String temp = "";` থাকে যা পরবর্তীতে কোনো কাজেই লাগেনি, সেগুলো মুছে দিন।

```
git checkout -b task-6
git add .
git commit -m "Task #6. Remove Temporary Variables:"
git checkout master
git merge task-6
```

Task 7: Eliminate 'done' Variable

Code: সার্চ অপশনের (?) লজিকটি এমনভাবে লিখুন:

```
String searchName = args[0].substring(1);
String[] employees = fileContent.split(", ");
for (String emp : employees) {
    if (emp.trim().equals(searchName)) {
        System.out.println("We found it!");
        return; // Exits directly, no 'done' variable needed
    }
}
System.out.println("Not found!");
```

```
git checkout -b task-7
git add .
git commit -m "Task #7. Eliminates the 'done' Control-Flow Variable. Adds Better Response for Search
Operation:"
git checkout master
git merge task-7
```

Task 8: Simplify Count Operation

Code: c আর্গুমেন্টের জন্য ক্যারেক্টার লুপ না চালিয়ে সহজভাবে লিখুন:

```
String fileContent = readFromFile(Constants.FILE_NAME);
String[] employees = fileContent.split(", ");
```

```
System.out.println(employees.length + " word(s) found ");
```

```
git checkout -b task-8
git add .
git commit -m "Task #8. Simplifies the Logic Behind the Count Operation:"
git checkout master
git merge task-8
```

Task 9: Handle Invalid Arguments

Code: `if-else` বা `switch` এর শেষে ডিফল্ট কেস দিন:

```
else {
    System.out.println("Error: Invalid argument passed! Use l, s, +, ?, c, or u.");
}
```

```
git checkout -b task-9
git add .
git commit -m "Task #9. Adds Handling for Case When User Enters Invalid Arguments:"
git checkout master
git merge task-9
```

Task 10: Comments & Naming

Code: ক্লাসের শুরুতে এবং মেথডগুলোর উপরে কমেন্ট দিন। যেমন:

```
// This class manages the employee list operations
// such as read, write, search and count.
```

```
git checkout -b task-10
git add .
git commit -m "Task #10. Add More Comments and Makes More Naming Improvements:"
git checkout master
git merge task-10
```

```
import java.io.*;
import java.util.*;

public class EmployeeManager {
    public static void main(String[] args) {
        // Add a check at the beginning to ensure the correct number of arguments
        if (args.length == 0) {
            System.out.println("Error: No arguments provided.");
            return;
        }

        // Check arguments
        if (args[0].equals("l")) {
            System.out.println("Loading data ...");
            try {
                BufferedReader r = new BufferedReader(
                        new InputStreamReader(
                                new FileInputStream("employees.txt")));
                String l = r.readLine();
                String e[] = l.split(",");
                for (String emp : e) {
                    System.out.println(emp);
                }
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].equals("s")) {
            System.out.println("Loading data ...");
            try {
                BufferedReader r = new BufferedReader(
                        new InputStreamReader(
                                new FileInputStream("employees.txt")));
                String l = r.readLine();
                System.out.println(l);
                String e[] = l.split(",");
                Random rand = new Random();
                int idx = rand.nextInt(e.length);
                System.out.println(e[idx]);
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].contains("+")) {
            System.out.println("Loading data ...");
            try {
```

```
            BufferedWriter w = new BufferedWriter(
                    new FileWriter("employees.txt", true));
            String n = args[0].substring(1);
            w.write(", " + n);
            w.close();
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].contains("?")) {
        System.out.println("Loading data ...");
        try {
            BufferedReader r = new BufferedReader(
                    new InputStreamReader(
                            new FileInputStream("employees.txt")));
            String l = r.readLine();
            String e[] = l.split(",");
            boolean found = false;
            String s = args[0].substring(1);
            for (int i = 0; i < e.length && !found; i++) {
                if (e[i].equals(s)) {
                    System.out.println("Employee found!");
                    found = true;
                }
            }
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].contains("c")) {
        System.out.println("Loading data ...");
        try {
            BufferedReader r = new BufferedReader(
                    new InputStreamReader(
                            new FileInputStream("employees.txt")));
            String l = r.readLine();
            char[] chars = l.toCharArray();
            boolean inWord = false;
            int count = 0;
            for (char c : chars) {
                if (c == ' ') {
                    if (!inWord) {
                        count++;
                        inWord = true;
                    } else {
                        inWord = false;
```

```
                }
            }
        }
        System.out.println(count + " word(s) found " + chars.length);
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("u")) {
    System.out.println("Loading data ...");
    try {
        BufferedReader r = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream("employees.txt")));
        String l = r.readLine();
        String e[] = l.split(",");
        String n = args[0].substring(1);
        for (int i = 0; i < e.length; i++) {
            if (e[i].equals(n)) {
                e[i] = "Updated";
            }
        }
        BufferedWriter w = new BufferedWriter(
                new FileWriter("employees.txt"));
        w.write(String.join(",", e));
        w.close();
    } catch (Exception e) {
    }
    System.out.println("Data Updated.");
} else if (args[0].contains("d")) {
    System.out.println("Loading data ...");
    try {
        BufferedReader r = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream("employees.txt")));
        String l = r.readLine();
        String e[] = l.split(",");
        String n = args[0].substring(1);
        List<String> list = new ArrayList<>(Arrays.asList(e));
        list.remove(n);
        BufferedWriter w = new BufferedWriter(
                new FileWriter("employees.txt"));
        w.write(String.join(",", list));
        w.close();
    } catch (Exception e) {
```

```
            }
            System.out.println("Data Deleted.");
        }
    }
}
```

```
import java.io.*;
import java.util.*;

public class EmployeeManager {
    public static void main(String[] args) {
        // Add a check at the beginning to ensure the correct number of arguments
        if (args.length == 0) {
            System.out.println("Error: No arguments provided. Please provide a valid argument
(e.g., l, s, +Name, ?Name).");
            return;
        }

        // Check arguments
        if (args[0].equals("l")) {
            System.out.println("Loading data ...");
            try {
                BufferedReader reader = new BufferedReader(
                        new InputStreamReader(
                                new FileInputStream("employees.txt")));
                String line = reader.readLine();
                String[] employees = line.split(",");
                for (String emp : employees) {
                    System.out.println(emp);
                }
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].equals("s")) {
            System.out.println("Loading data ...");
            try {
                BufferedReader reader = new BufferedReader(
                        new InputStreamReader(
                                new FileInputStream("employees.txt")));
                String line = reader.readLine();
                System.out.println(line);
                String[] employees = line.split(",");
                Random rand = new Random();
                int idx = rand.nextInt(employees.length);
                System.out.println(employees[idx]);
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].contains("+")) {
            System.out.println("Loading data ...");
```

```
    try {
        BufferedWriter writer = new BufferedWriter(
                new FileWriter("employees.txt", true));
        String newEmployee = args[0].substring(1);
        writer.write(", " + newEmployee);
        writer.close();
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("?")) {
    System.out.println("Loading data ...");
    try {
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream("employees.txt")));
        String line = reader.readLine();
        String[] employees = line.split(",");
        boolean found = false;
        String targetName = args[0].substring(1);
        for (int i = 0; i < employees.length && !found; i++) {
            if (employees[i].equals(targetName)) {
                System.out.println("Employee found!");
                found = true;
            }
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("c")) {
    System.out.println("Loading data ...");
    try {
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream("employees.txt")));
        String line = reader.readLine();
        char[] chars = line.toCharArray();
        boolean inWord = false;
        int count = 0;
        for (char character : chars) {
            if (character == ' ') {
                if (!inWord) {
                    count++;
                    inWord = true;
                } else {
```

```
                inWord = false;
            }
        }
    }
    System.out.println(count + " word(s) found " + chars.length);
  } catch (Exception e) {
  }
  System.out.println("Data Loaded.");
} else if (args[0].contains("u")) {
  System.out.println("Loading data ...");
  try {
    BufferedReader reader = new BufferedReader(
        new InputStreamReader(
            new FileInputStream("employees.txt")));
    String line = reader.readLine();
    String[] employees = line.split(",");
    String targetToUpdate = args[0].substring(1);
    for (int i = 0; i < employees.length; i++) {
      if (employees[i].equals(targetToUpdate)) {
        employees[i] = "Updated";
      }
    }
    BufferedWriter writer = new BufferedWriter(
        new FileWriter("employees.txt"));
    writer.write(String.join(",", employees));
    writer.close();
  } catch (Exception e) {
  }
  System.out.println("Data Updated.");
} else if (args[0].contains("d")) {
  System.out.println("Loading data ...");
  try {
    BufferedReader reader = new BufferedReader(
        new InputStreamReader(
            new FileInputStream("employees.txt")));
    String line = reader.readLine();
    String[] employees = line.split(",");
    String targetToDelete = args[0].substring(1);
    List<String> employeeList = new ArrayList<>(Arrays.asList(employees));
    employeeList.remove(targetToDelete);
    BufferedWriter writer = new BufferedWriter(
        new FileWriter("employees.txt"));
    writer.write(String.join(",", employeeList));
    writer.close();
```

```
            } catch (Exception e) {
            }
            System.out.println("Data Deleted.");
        }
    }
}
```

```
import java.io.*;
import java.util.*;

public class EmployeeManager {

    // Method to read data from the file
    public static String readEmployees() throws IOException {
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream("employees.txt")));
        String line = reader.readLine();
        reader.close();
        return line;
    }

    // Method to write data to the file (overwrite)
    public static void writeEmployees(String data) throws IOException {
        BufferedWriter writer = new BufferedWriter(
                new FileWriter("employees.txt"));
        writer.write(data);
        writer.close();
    }

    // Method to append data to the file
    public static void appendEmployee(String data) throws IOException {
        BufferedWriter writer = new BufferedWriter(
                new FileWriter("employees.txt", true));
        writer.write(data);
        writer.close();
    }

    public static void main(String[] args) {
        // Add a check at the beginning to ensure the correct number of arguments
        if (args.length == 0) {
            System.out.println("Error: No arguments provided. Please provide a valid argument
(e.g., l, s, +Name, ?Name).");
            return;
        }

        // Check arguments
        if (args[0].equals("l")) {
            System.out.println("Loading data ...");
            try {
                String line = readEmployees();
```

```
            String[] employees = line.split(",");
            for (String emp : employees) {
                System.out.println(emp);
            }
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].equals("s")) {
        System.out.println("Loading data ...");
        try {
            String line = readEmployees();
            System.out.println(line);
            String[] employees = line.split(",");
            Random rand = new Random();
            int idx = rand.nextInt(employees.length);
            System.out.println(employees[idx]);
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].contains("+")) {
        System.out.println("Loading data ...");
        try {
            String newEmployee = args[0].substring(1);
            appendEmployee(", " + newEmployee);
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].contains("?")) {
        System.out.println("Loading data ...");
        try {
            String line = readEmployees();
            String[] employees = line.split(",");
            boolean found = false;
            String targetName = args[0].substring(1);
            for (int i = 0; i < employees.length && !found; i++) {
                if (employees[i].equals(targetName)) {
                    System.out.println("Employee found!");
                    found = true;
                }
            }
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].contains("c")) {
```

```
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        char[] chars = line.toCharArray();
        boolean inWord = false;
        int count = 0;
        for (char character : chars) {
            if (character == ' ') {
                if (!inWord) {
                    count++;
                    inWord = true;
                } else {
                    inWord = false;
                }
            }
        }
        System.out.println(count + " word(s) found " + chars.length);
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("u")) {
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        String[] employees = line.split(",");
        String targetToUpdate = args[0].substring(1);
        for (int i = 0; i < employees.length; i++) {
            if (employees[i].equals(targetToUpdate)) {
                employees[i] = "Updated";
            }
        }
        writeEmployees(String.join(",", employees));
    } catch (Exception e) {
    }
    System.out.println("Data Updated.");
} else if (args[0].contains("d")) {
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        String[] employees = line.split(",");
        String targetToDelete = args[0].substring(1);
        List<String> employeeList = new ArrayList<>(Arrays.asList(employees));
        employeeList.remove(targetToDelete);
        writeEmployees(String.join(",", employeeList));
```

```
            } catch (Exception e) {
            }
            System.out.println("Data Deleted.");
        }
    }
}
```

Constants.java (New)

```
public class Constants {
    // Constant for the employee file path
    public static final String EMPLOYEE_FILE_PATH = "employees.txt";

}
```

Main file:

```
import java.io.*;
import java.util.*;

public class EmployeeManager {

    // Method to read data from the file
    public static String readEmployees() throws IOException {
        // Task #5 Update
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream(Constants.EMPLOYEE_FILE_PATH)));
        String line = reader.readLine();
        reader.close();
        return line;
    }

    // Method to write data to the file (overwrite)
    public static void writeEmployees(String data) throws IOException {
        // Task #5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH));
        writer.write(data);
        writer.close();
    }

    // Method to append data to the file
    public static void appendEmployee(String data) throws IOException {
        // Task #5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH, true));
```

```
        writer.write(data);
        writer.close();
    }

    public static void main(String[] args) {
        // Add a check at the beginning to ensure the correct number of arguments
        if (args.length == 0) {
            System.out.println("Error: No arguments provided. Please provide a valid argument
(e.g., l, s, +Name, ?Name).");
            return;
        }

        // Check arguments
        if (args[0].equals("l")) {
            System.out.println("Loading data ...");
            try {
                String line = readEmployees();
                String[] employees = line.split(",");
                for (String emp : employees) {
                    System.out.println(emp);
                }
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].equals("s")) {
            System.out.println("Loading data ...");
            try {
                String line = readEmployees();
                System.out.println(line);
                String[] employees = line.split(",");
                Random rand = new Random();
                int idx = rand.nextInt(employees.length);
                System.out.println(employees[idx]);
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].contains("+")) {
            System.out.println("Loading data ...");
            try {
                String newEmployee = args[0].substring(1);
                appendEmployee(", " + newEmployee);
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
```

```
} else if (args[0].contains("?")) {
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        String[] employees = line.split(",");
        boolean found = false;
        String targetName = args[0].substring(1);
        for (int i = 0; i < employees.length && !found; i++) {
            if (employees[i].equals(targetName)) {
                System.out.println("Employee found!");
                found = true;
            }
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("c")) {
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        char[] chars = line.toCharArray();
        boolean inWord = false;
        int count = 0;
        for (char character : chars) {
            if (character == ' ') {
                if (!inWord) {
                    count++;
                    inWord = true;
                } else {
                    inWord = false;
                }
            }
        }
        System.out.println(count + " word(s) found " + chars.length);
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("u")) {
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        String[] employees = line.split(",");
        String targetToUpdate = args[0].substring(1);
        for (int i = 0; i < employees.length; i++) {
```

```
                if (employees[i].equals(targetToUpdate)) {
                    employees[i] = "Updated";
                }
            }
            writeEmployees(String.join(",", employees));
        } catch (Exception e) {
        }
        System.out.println("Data Updated.");
    } else if (args[0].contains("d")) {
        System.out.println("Loading data ...");
        try {
            String line = readEmployees();
            String[] employees = line.split(",");
            String targetToDelete = args[0].substring(1);
            List<String> employeeList = new ArrayList<>(Arrays.asList(employees));
            employeeList.remove(targetToDelete);
            writeEmployees(String.join(",", employeeList));
        } catch (Exception e) {
        }
        System.out.println("Data Deleted.");
    }
  }
}
```

```
import java.io.*;
import java.util.*;

public class EmployeeManager {

    // Method to read data from the file
    public static String readEmployees() throws IOException {
        // Task #5 Update
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream(Constants.EMPLOYEE_FILE_PATH)));
        String line = reader.readLine();
        reader.close();
        return line;
    }

    // Method to write data to the file (overwrite)
    public static void writeEmployees(String data) throws IOException {
        // Task #5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH));
        writer.write(data);
        writer.close();
    }

    // Method to append data to the file
    public static void appendEmployee(String data) throws IOException {
        // Task #5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH, true));
        writer.write(data);
        writer.close();
    }

    public static void main(String[] args) {
        // Add a check at the beginning to ensure the correct number of arguments
        if (args.length == 0) {
            System.out.println("Error: No arguments provided. Please provide a valid argument
(e.g., l, s, +Name, ?Name).");
            return;
        }

        // Check arguments
        if (args[0].equals("l")) {
```

```
            System.out.println("Loading data ...");
            try {
                // Task #6 Update: Removed temporary variables 'line' and 'employees'.
                // Chaining readEmployees() directly with split().
                for (String emp : readEmployees().split(",")) {
                    System.out.println(emp);
                }
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].equals("s")) {
            System.out.println("Loading data ...");
            try {
                String line = readEmployees();
                System.out.println(line);
                String[] employees = line.split(",");
                // Task #6 Update: Removed temporary variables 'rand' and 'idx'.
                // Creating Random instance and getting index in a single line.
                System.out.println(employees[new Random().nextInt(employees.length)]);
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].contains("+")) {
            System.out.println("Loading data ...");
            try {
                // Task #6 Update: Removed temporary variable 'newEmployee'.
                // Passing the substring directly to the method.
                appendEmployee(", " + args[0].substring(1));
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].contains("?")) {
            System.out.println("Loading data ...");
            try {
                // Task #6 Update: Removed temporary variables 'line', 'employees', 'found', and
'targetName'.
                // Iterating directly over the split array and using 'break' instead of a boolean flag.
                for (String emp : readEmployees().split(",")) {
                    if (emp.equals(args[0].substring(1))) {
                        System.out.println("Employee found!");
                        break;
                    }
                }
            } catch (Exception e) {
```

```
            }
            System.out.println("Data Loaded.");
        } else if (args[0].contains("c")) {
            System.out.println("Loading data ...");
            try {
                // Task #6 Update: Removed temporary variable 'line'.
                // Directly converting the result of readEmployees() to a char array.
                char[] chars = readEmployees().toCharArray();
                boolean inWord = false;
                int count = 0;
                for (char character : chars) {
                    if (character == ' ') {
                        if (!inWord) {
                            count++;
                            inWord = true;
                        } else {
                            inWord = false;
                        }
                    }
                }
                System.out.println(count + " word(s) found " + chars.length);
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (args[0].contains("u")) {
            System.out.println("Loading data ...");
            try {
                // Task #6 Update: Removed temporary variables 'line' and 'targetToUpdate'.
                // Checking the condition directly against args[0].substring(1).
                String[] employees = readEmployees().split(",");
                for (int i = 0; i < employees.length; i++) {
                    if (employees[i].equals(args[0].substring(1))) {
                        employees[i] = "Updated";
                    }
                }
                writeEmployees(String.join(",", employees));
            } catch (Exception e) {
            }
            System.out.println("Data Updated.");
        } else if (args[0].contains("d")) {
            System.out.println("Loading data ...");
            try {
                // Task #6 Update: Removed temporary variables 'line', 'employees', and
'targetToDelete'.
```

```
            // Creating the list directly and removing the item directly.
            List<String> employeeList = new
ArrayList<>(Arrays.asList(readEmployees().split(",")));
            employeeList.remove(args[0].substring(1));
            writeEmployees(String.join(",", employeeList));
        } catch (Exception e) {
        }
        System.out.println("Data Deleted.");
    }
  }
}
```

```
import java.io.*;
import java.util.*;

public class EmployeeManager {

    // Method to read data from the file
    public static String readEmployees() throws IOException {
        // Task 5 Update
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream(Constants.EMPLOYEE_FILE_PATH)));
        String line = reader.readLine();
        reader.close();
        return line;
    }

    // Method to write data to the file (overwrite)
    public static void writeEmployees(String data) throws IOException {
        // Task 5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH));
        writer.write(data);
        writer.close();
    }

    // Method to append data to the file
    public static void appendEmployee(String data) throws IOException {
        // Task 5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH, true));
        writer.write(data);
        writer.close();
    }

    public static void main(String[] args) {
        // Add a check at the beginning to ensure the correct number of arguments
        if (args.length == 0) {
            System.out.println("Error: No arguments provided. Please provide a valid argument
(e.g., l, s, +Name, ?Name).");
            return;
        }

        // Check arguments
        if (args[0].equals("l")) {
```

```
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
        for (String emp : readEmployees().split(",")) {
            System.out.println(emp);
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].equals("s")) {
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        System.out.println(line);
        String[] employees = line.split(",");
        // Task 6 Update
        System.out.println(employees[new Random().nextInt(employees.length)]);
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("+")) {
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
        appendEmployee(", " + args[0].substring(1));
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("?")) {
    System.out.println("Loading data ...");
    try {
        // Task 7 Update
        if (Arrays.asList(readEmployees().split(",")).contains(args[0].substring(1))) {
            System.out.println("Employee '" + args[0].substring(1) + "' found!");
        } else {
            System.out.println("Employee '" + args[0].substring(1) + "' not found!");
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("c")) {
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
```

```
            char[] chars = readEmployees().toCharArray();
            boolean inWord = false;
            int count = 0;
            for (char character : chars) {
                if (character == ' ') {
                    if (!inWord) {
                        count++;
                        inWord = true;
                    } else {
                        inWord = false;
                    }
                }
            }
            System.out.println(count + " word(s) found " + chars.length);
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].contains("u")) {
        System.out.println("Loading data ...");
        try {
            // Task 6 Update
            String[] employees = readEmployees().split(",");
            for (int i = 0; i < employees.length; i++) {
                if (employees[i].equals(args[0].substring(1))) {
                    employees[i] = "Updated";
                }
            }
            writeEmployees(String.join(",", employees));
        } catch (Exception e) {
        }
        System.out.println("Data Updated.");
    } else if (args[0].contains("d")) {
        System.out.println("Loading data ...");
        try {
            // Task 6 Update
            List<String> employeeList = new
ArrayList<>(Arrays.asList(readEmployees().split(",")));
            employeeList.remove(args[0].substring(1));
            writeEmployees(String.join(",", employeeList));
        } catch (Exception e) {
        }
        System.out.println("Data Deleted.");
    }
  }
```

```
}
```

```
import java.io.*;
import java.util.*;

public class EmployeeManager {

    // Method to read data from the file
    public static String readEmployees() throws IOException {
        // Task 5 Update
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream(Constants.EMPLOYEE_FILE_PATH)));
        String line = reader.readLine();
        reader.close();
        return line;
    }

    // Method to write data to the file (overwrite)
    public static void writeEmployees(String data) throws IOException {
        // Task 5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH));
        writer.write(data);
        writer.close();
    }

    // Method to append data to the file
    public static void appendEmployee(String data) throws IOException {
        // Task 5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH, true));
        writer.write(data);
        writer.close();
    }

    public static void main(String[] args) {
        // Add a check at the beginning to ensure the correct number of arguments
        if (args.length == 0) {
            System.out.println("Error: No arguments provided. Please provide a valid argument
(e.g., l, s, +Name, ?Name).");
            return;
        }

        // Check arguments
        if (args[0].equals("l")) {
```

```
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
        for (String emp : readEmployees().split(",")) {
            System.out.println(emp);
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].equals("s")) {
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        System.out.println(line);
        String[] employees = line.split(",");
        // Task 6 Update
        System.out.println(employees[new Random().nextInt(employees.length)]);
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("+")) {
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
        appendEmployee(", " + args[0].substring(1));
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("?")) {
    System.out.println("Loading data ...");
    try {
        // Task 7 Update
        if (Arrays.asList(readEmployees().split(",")).contains(args[0].substring(1))) {
            System.out.println("Employee '" + args[0].substring(1) + "' found!");
        } else {
            System.out.println("Employee '" + args[0].substring(1) + "' not found!");
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("c")) {
    System.out.println("Loading data ...");
    try {
        // Task 8 Update
```

```
            String text = readEmployees();
            System.out.println(text.split(",").length + " word(s) found " + text.length());
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].contains("u")) {
        System.out.println("Loading data ...");
        try {
            // Task 6 Update
            String[] employees = readEmployees().split(",");
            for (int i = 0; i < employees.length; i++) {
                if (employees[i].equals(args[0].substring(1))) {
                    employees[i] = "Updated";
                }
            }
            writeEmployees(String.join(",", employees));
        } catch (Exception e) {
        }
        System.out.println("Data Updated.");
    } else if (args[0].contains("d")) {
        System.out.println("Loading data ...");
        try {
            // Task 6 Update
            List<String> employeeList = new
ArrayList<>(Arrays.asList(readEmployees().split(",")));
            employeeList.remove(args[0].substring(1));
            writeEmployees(String.join(",", employeeList));
        } catch (Exception e) {
        }
        System.out.println("Data Deleted.");
    }
  }
}
```

```
import java.io.*;
import java.util.*;

public class EmployeeManager {

    // Method to read data from the file
    public static String readEmployees() throws IOException {
        // Task 5 Update
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream(Constants.EMPLOYEE_FILE_PATH)));
        String line = reader.readLine();
        reader.close();
        return line;
    }

    // Method to write data to the file (overwrite)
    public static void writeEmployees(String data) throws IOException {
        // Task 5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH));
        writer.write(data);
        writer.close();
    }

    // Method to append data to the file
    public static void appendEmployee(String data) throws IOException {
        // Task 5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH, true));
        writer.write(data);
        writer.close();
    }

    public static void main(String[] args) {
        // Add a check at the beginning to ensure the correct number of arguments
        if (args.length == 0) {
            System.out.println("Error: No arguments provided. Please provide a valid argument
(e.g., l, s, +Name, ?Name, c, uName, dName).");
            return;
        }

        // Check arguments
        if (args[0].equals("l")) {
```

```
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
        for (String emp : readEmployees().split(",")) {
            System.out.println(emp);
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].equals("s")) {
    System.out.println("Loading data ...");
    try {
        String line = readEmployees();
        System.out.println(line);
        String[] employees = line.split(",");
        // Task 6 Update
        System.out.println(employees[new Random().nextInt(employees.length)]);
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("+")) {
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
        appendEmployee(", " + args[0].substring(1));
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("?")) {
    System.out.println("Loading data ...");
    try {
        // Task 7 Update
        if (Arrays.asList(readEmployees().split(",")).contains(args[0].substring(1))) {
            System.out.println("Employee '" + args[0].substring(1) + "' found!");
        } else {
            System.out.println("Employee '" + args[0].substring(1) + "' not found!");
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (args[0].contains("c")) {
    System.out.println("Loading data ...");
    try {
        // Task 8 Update
```

```
            String text = readEmployees();
            System.out.println(text.split(",").length + " word(s) found " + text.length());
        } catch (Exception e) {
        }
        System.out.println("Data Loaded.");
    } else if (args[0].contains("u")) {
        System.out.println("Loading data ...");
        try {
            // Task 6 Update
            String[] employees = readEmployees().split(",");
            for (int i = 0; i < employees.length; i++) {
                if (employees[i].equals(args[0].substring(1))) {
                    employees[i] = "Updated";
                }
            }
            writeEmployees(String.join(",", employees));
        } catch (Exception e) {
        }
        System.out.println("Data Updated.");
    } else if (args[0].contains("d")) {
        System.out.println("Loading data ...");
        try {
            // Task 6 Update
            List<String> employeeList = new
ArrayList<>(Arrays.asList(readEmployees().split(",")));
            employeeList.remove(args[0].substring(1));
            writeEmployees(String.join(",", employeeList));
        } catch (Exception e) {
        }
        System.out.println("Data Deleted.");
    } else {
        // Task 9 Update
        System.out.println("Error: Invalid argument '" + args[0] + "'. Please provide a valid
argument.");
    }
  }
}
```

```
import java.io.*;
import java.util.*;

public class EmployeeManager {

    // Task 10 Update: rename method for getting all emp data
    public static String getAllEmployees() throws IOException {
        // Task 5 Update
        BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                        new FileInputStream(Constants.EMPLOYEE_FILE_PATH)));
        String line = reader.readLine();
        reader.close();
        return line;
    }

    // Task 10 Update: method to save all data into file
    public static void saveAllEmployees(String data) throws IOException {
        // Task 5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH));
        writer.write(data);
        writer.close();
    }

    // Task 10 Update: method for adding new emplyee
    public static void addNewEmployee(String data) throws IOException {
        // Task 5 Update
        BufferedWriter writer = new BufferedWriter(
                new FileWriter(Constants.EMPLOYEE_FILE_PATH, true));
        writer.write(data);
        writer.close();
    }

    public static void main(String[] args) {
        // Task 10 Update: cheking if user not give any argument
        if (args.length == 0) {
            System.out.println("Error: No arguments provided. Please provide a valid argument
(e.g., l, s, +Name, ?Name, c, uName, dName).");
            return;
        }

        // Task 10 Update: taking the fast argument in a variable
        String userCmd = args[0];
```

```
// Check arguments
if (userCmd.equals("l")) {
    // Task 10 Update: this is loading oparetion for show all emp
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
        for (String emp : getAllEmployees().split(",")) {
            System.out.println(emp);
        }
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (userCmd.equals("s")) {
    // Task 10 Update: searching a random emplyee from file
    System.out.println("Loading data ...");
    try {
        String line = getAllEmployees();
        System.out.println(line);
        String[] employees = line.split(",");
        // Task 6 Update
        System.out.println(employees[new Random().nextInt(employees.length)]);
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (userCmd.contains("+")) {
    // Task 10 Update: add new emplye to the file
    System.out.println("Loading data ...");
    try {
        // Task 6 Update
        addNewEmployee(", " + userCmd.substring(1));
    } catch (Exception e) {
    }
    System.out.println("Data Loaded.");
} else if (userCmd.contains("?")) {
    // Task 10 Update: find if a employe is exist or not
    System.out.println("Loading data ...");
    try {
        // Task 7 Update
        if (Arrays.asList(getAllEmployees().split(",")).contains(userCmd.substring(1))) {
            System.out.println("Employee '" + userCmd.substring(1) + "' found!");
        } else {
            System.out.println("Employee '" + userCmd.substring(1) + "' not found!");
        }
```

```
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (userCmd.contains("c")) {
            // Task 10 Update: count how many words is in there
            System.out.println("Loading data ...");
            try {
                // Task 8 Update
                String text = getAllEmployees();
                System.out.println(text.split(",").length + " word(s) found " + text.length());
            } catch (Exception e) {
            }
            System.out.println("Data Loaded.");
        } else if (userCmd.contains("u")) {
            // Task 10 Update: update a emplyee name to Updated
            System.out.println("Loading data ...");
            try {
                // Task 6 Update
                String[] employees = getAllEmployees().split(",");
                for (int i = 0; i < employees.length; i++) {
                    if (employees[i].equals(userCmd.substring(1))) {
                        employees[i] = "Updated";
                    }
                }
                saveAllEmployees(String.join(",", employees));
            } catch (Exception e) {
            }
            System.out.println("Data Updated.");
        } else if (userCmd.contains("d")) {
            // Task 10 Update: delet a employee form the list
            System.out.println("Loading data ...");
            try {
                // Task 6 Update
                List<String> employeeList = new
ArrayList<>(Arrays.asList(getAllEmployees().split(",")));
                employeeList.remove(userCmd.substring(1));
                saveAllEmployees(String.join(",", employeeList));
            } catch (Exception e) {
            }
            System.out.println("Data Deleted.");
        } else {
            // Task 9 Update
            System.out.println("Error: Invalid argument '" + userCmd + "'. Please provide a valid
argument.");
```

```
    }
  }
}
```

https://github.com/afsanulislam/CSE2142-2026-G1